Extending zkVoting: Implementing Missing Protocol Components and Novel Security Improvements Over the Original Paper

Ayush
Department of Computer Science and Engineering
Indian Institute of Technology Patna
Patna, India

Durjoy Majumdar
Department of Computer Science and Engineering
Indian Institute of Technology Patna
Patna, India

***Abstract*—The zkVoting paper by Park et al. [1] introduces a coercion-resistant, end-to-end (E2E) verifiable e-voting system centred on a novel *nullifiable commitment scheme*. Despite providing rigorous security proofs and performance benchmarks, the accompanying prototype implementation omits the paper's core cryptographic contributions entirely—the nullifiable commitment scheme, real/fake casting key issuance, key deniability (SimKeyProve), Merkle-tree eligibility verification, and the O(n) nullification-based tally. Furthermore, three significant limitations remain unaddressed in the paper itself: (i) the absence of the π_{total} eligibility proof that prevents ballot stuffing; (ii) a single-authority trust assumption for the master secret key that collapses coercion resistance if the authority is compromised; and (iii) cross-election vote linkability arising from the reuse of casting keys.**

In this work we make two contributions. First, we implement all missing components from the original paper: Construction 1 (Baby Jubjub elliptic-curve nullifiable commitment), real/fake key registration with KeyProve and SimKeyProve, a Poseidon Merkle tree for the casting-key set, an enhanced Circom circuit encoding the full R_{vote} relation, and O(n) tally-time nullification. Second, we conduct a systematic audit identifying 13 security assumptions in the original protocol, and implement six novel improvements addressing five of them: (1) a concrete π_{total} proof using the homomorphic property of the nullifiable commitment scheme and a DLEQ Schnorr argument; (2) (t, n)-Shamir secret sharing of the master secret key with Feldman VSS verification and threshold partial-decryption tally; (3) election-scoped key derivation for cross-election unlinkability; (4) ephemeral epoch secrets for forward secrecy against retroactive deanonymisation; (5) commit-reveal registration for censorship resistance; and (6) a hidden re-vote grace period to prevent timing-based coercion attacks. We provide a detailed comparative security and performance analysis between the original protocol and our enhanced system.

***Index Terms*—zero-knowledge proofs, e-voting, coercion resistance, nullifiable commitment, Shamir secret sharing, Baby Jubjub, Circom, snarkjs**

I. Introduction

Electronic voting systems must simultaneously guarantee *ballot privacy*, *coercion resistance*, and *end-to-end (E2E) verifiability*—three properties that are difficult to achieve jointly. The zkVoting paper [1] addresses this challenge with a novel cryptographic primitive, the *nullifiable commitment scheme*, and an E2E verifiable protocol based on zk-SNARKs. The scheme's central insight is the *fake-keys* approach: voters receive one real and arbitrarily many indistinguishable fake casting keys; coercers cannot distinguish real from fake even when voters reveal keys and provide proofs.

Despite its theoretical contributions, the publicly available prototype implements only a narrow subset of the described protocol. The complete nullifiable commitment scheme (Construction 1), the Merkle-tree eligibility set, the R_{vote} circuit constraints for voter-key binding, and the nullification-based tally are all absent. Additionally, we identify three protocol-level limitations in the paper itself that undermine its security claims in practice.

A. Contributions

1) **Complete implementation** of all cryptographic components described in Sections 2–4 of the zkVoting paper, using Baby Jubjub EC arithmetic (circomlibjs), Poseidon hashing, and Groth16 zk-SNARKs.
2) **Systematic assumption audit**: identification and categorisation of 13 security assumptions in the original protocol, ranked by severity and real-world impact (Section VI).
3) π_{total} **eligibility proof** (Improvement I): a concrete non-interactive DLEQ Schnorr proof over the homomorphically aggregated commitment that proves exactly n_v real casting keys were issued, preventing ballot stuffing.
4) **Threshold master secret key** (Improvement II): (t, n)-Shamir secret sharing of msk with Feldman VSS verification and a threshold partial-nullification tally protocol, eliminating the single-authority trust bottleneck.
5) **Cross-election unlinkability** (Improvement III): election-scoped key derivation $ckHash_e = H(ckHash, e)$ prevents linking a voter's participation across elections.
6) **Forward secrecy** (Improvement IV): per-election ephemeral secrets in the serial number derivation prevent

retroactive deanonymisation even if the voter's master secret is later compromised.

7) **Registration censorship resistance** (Improvement V): commit-reveal registration with a public audit trail makes authority censorship cryptographically detectable.
8) **Re-vote grace period** (Improvement VI): a hidden grace window after the public deadline allows coerced voters to re-vote, closing timing-based coercion attacks.
9) **Comparative analysis** of security properties, constraint counts, and API surface between the original paper, the basic prototype, and our enhanced system.

II. Background: the zkVoting Protocol

A. Nullifiable Commitment Scheme (Construction 1)

The nullifiable commitment scheme Π_{NC} comprises eight algorithms: Setup, KeyGen, Commit, Nullify, Open, $\mathsf{Open}_{\mathrm{null}}$, KeyProve, and SimKeyProve. It operates over a prime-order elliptic curve group $\mathbb{G}$ with order q (Baby Jubjub). The concrete instantiation is:

Setup. $\rho \leftarrow\$\ \mathbb{Z}_q^*;\ \ g_1, g_2, g_3 \leftarrow\$\ \mathbb{G};\ \ g_4 = \rho \cdot g_1;$

$$mpk = (g_1, g_2, g_3, g_4);\ msk = \rho$$

$ck^* = (g_2 - g_4,\ g_3)$.

KeyGen. $h_1 \leftarrow\$\ \mathbb{G};\ h_2 = msk \cdot h_1 + b \cdot g_3\ (b \in \{0,1\});\ ck = (h_1, h_2)$.

Commit. $C_1 = r \cdot g_1 + m \cdot h_1;\ C_2 = r \cdot g_2 + m \cdot h_2;\ cm = (C_1, C_2)$.

Nullify. $cm^* = C_2 - msk \cdot C_1$. For real keys ($b = 1$): opens to m via $\mathsf{Open}_{\mathrm{null}}$. For fake keys ($b = 0$): opens to 0.

B. E2E Verifiable Voting Protocol

The four phases are Setup, Register, Vote, and Tally. The voter's ballot is:

$$B = \big(e,\ sn,\ CT,\ cm,\ \pi_{\text{vote}}\big)$$

where $sn = H(e, ck, sk_{\mathrm{id}})$, cm is the nullifiable commitment, CT is the hybrid encryption of the message, and π_{vote} is a Groth16 proof for R_{vote} (see Section IV-D).

C. Security Properties

The paper proves ballot privacy (BP), voter anonymity (VA), receipt freeness (RF), coercion resistance (CR), individual verifiability (IV), universal verifiability (UV), E2E verifiability (E2E-V), and eligibility verifiability (EV) under standard assumptions (DL, DDH, collision-resistance, zk-SNARK soundness).

III. What the Basic Prototype Was Missing

The original prototype implemented only three constraints in its Circom circuit:

1) $vote \in candidateList$
2) $sn = \mathrm{Poseidon}(electionId,\ castingKey,\ voterSecret)$ where $castingKey$ was a single random scalar—not an EC key.
3) $cm = \mathrm{Poseidon}(vote, voteSalt)$—a plain Poseidon commitment, not a nullifiable one.

The following components from the paper were completely absent:

- No elliptic-curve nullifiable commitment (Construction 1).
- No real/fake key distinction at registration—only one key per voter.
- No KeyProve or SimKeyProve (key deniability absent).
- No Merkle tree for the casting-key set rt.
- No R_{id} constraint ($pkid = H(sk_{\mathrm{id}})$).
- No Merkle membership proof in the circuit.
- Tally by plaintext decryption only—no nullification, so fake-key ballots would be counted as real votes.
- No π_{total} eligibility proof (not even described concretely).

IV. Our Implementation of Missing Components

A. Nullifiable Commitment Scheme

We implement Construction 1 in `backend/src/lib/nullifiableCommitment.js` using the `circomlibjs` Baby Jubjub API. Elliptic-curve points are represented as `Uint8Array` field elements in Montgomery form and serialised to decimal strings for JSON persistence.

Key implementation details:

- Scalar field: $q \approx 2^{251}$ (Baby Jubjub suborder).
- Random scalars sampled via rejection sampling from 256-bit entropy.
- Point negation (twisted-Edwards): $-(x, y) = (-x, y)$.
- KeyProve: non-interactive Sigma protocol. Challenge c is stored in the proof tuple (p, k, c) without a Fiat-Shamir hash so that SimKeyProve transcripts are indistinguishable (key deniability).

Correctness was verified: real commitments nullify to (m, r), fake commitments nullify to $(0, r)$.

B. Real/Fake Key Registration

`POST /api/register` now invokes $\mathsf{KeyGen}(mpk, msk, 1)$ and issues a KeyProve proof. `POST /api/register/fake-key` issues $b = 0$ keys (unlimited). `POST /api/register/sim-key-prove` runs SimKeyProve: a voter under coercion can claim any key is "real" using a simulated proof indistinguishable from a genuine KeyProve.

C. Poseidon Merkle Tree

`backend/src/lib/merkleTree.js` implements a fixed-depth-16 ($2^{16} = 65{,}536$ voters) binary Merkle tree over Poseidon hashes. Leaf $i = \mathrm{Poseidon}(ckHash_i, pkid_i)$. Empty leaves use $\mathrm{Poseidon}(0, 0)$. Membership proofs are (path elements, path indices) pairs of length 16.

D. Enhanced Circom Circuit (R_{vote})

The new circuit `circuits/enhanced_vote.circom` encodes six constraints compared to the original three:

1) $pkid = \text{Poseidon}(voterSecret)$ [NEW]
2) $ckHash = \text{Poseidon}(h_{1x}, h_{1y}, h_{2x}, h_{2y})$ [NEW]
3) $sn = \text{Poseidon}(electionId, ckHash, voterSecret)$
4) $vote \in candidateList$
5) $cm = \text{Poseidon}(vote, voteSalt)$
6) Merkle path check: leaf $= \text{Poseidon}(ckHash, pkid)$ lies in tree with root $merkleRoot$ [NEW]

Public signals (10 total): $electionId$, $candidateList[0..4]$, $serialNumber$, $voteCommitment$, $publicKey$, $merkleRoot$.

E. Nullification-Based Tally

`GET /api/results/tally` follows Construction 2 (§4.4 of the paper): for each ballot (C_1, C_2), compute $cm^* = C_2 - msk \cdot C_1$, then verify $\mathsf{Open}_{\text{null}}(ck^*, cm^*, m, r)$. Ballots cast with fake keys nullify to zero and are discarded. Complexity: $O(n)$.

V. Novel Improvements Over the Paper

A. Improvement I: π_{total} Eligibility Proof

Limitation in the paper. Section 4.2 states that the authority must publish a proof π_{total} showing that the nullification of the aggregated commitment of all casting keys opens to n_v (the number of eligible voters):

$$R_{\text{total}} = \{(ck^*, cm, n_v\,;\, msk)\}$$

where $cm^* \leftarrow \mathsf{Nullify}(msk, cm)$ and $\mathsf{Open}_{\text{null}}(ck^*, cm^*, n_v, 0) = 1$. This prevents the authority from secretly issuing extra real keys to stuff the ballot box. The paper mentions 2,243 constraints for R_{total} (Table 3) but provides no concrete implementation.

Our implementation. We exploit the homomorphic property directly. For each issued casting key ck_i, we compute:

$$cm_i = \mathsf{Commit}(ck_i, 1;\, 0) = (g_1^0 \cdot h_{1,i}^1,\ g_2^0 \cdot h_{2,i}^1) = (h_{1,i},\ h_{2,i})$$

The aggregate: $cm_{\text{agg}} = \sum_i cm_i$. After nullification:

$$\begin{aligned} cm^*_{\text{agg}} &= C_{2,\text{agg}} - msk \cdot C_{1,\text{agg}} \\ &= \textstyle\sum_i h_{2,i} - msk \cdot \sum_i h_{1,i} \\ &= \textstyle\sum_i (h_{2,i} - msk \cdot h_{1,i}) \\ &= \textstyle\sum_i b_i \cdot g_3 = n_v^{\text{real}} \cdot g_3 \end{aligned}$$

where $n_v^{\text{real}} = \sum_i b_i$ counts only real keys. This opens to n_v^{real} under $ck^* = (g_2 - g_4, g_3)$ with $r = 0$.

We prove knowledge of msk via a non-interactive DLEQ Schnorr argument (Fiat-Shamir). Sample $t \leftarrow\$\ \mathbb{Z}_q$, then:

$$\begin{aligned} A_1 = t \cdot g_1, \quad A_2 &= -t \cdot C_{1,\text{agg}} \\ c &= H(g_1,\, g_4,\, C_{1,\text{agg}},\, cm^*_{\text{agg}}, \\ &\qquad A_1,\, A_2,\, n_v) \\ k &= t + msk \cdot c \bmod q \end{aligned}$$

Verifier checks: $k \cdot g_1 = A_1 + c \cdot g_4$ and $-k \cdot C_{1,\text{agg}} + C_{2,\text{agg}} - cm^*_{\text{agg}} = A_2 + c \cdot (C_{2,\text{agg}} - cm^*_{\text{agg}})$.

Exposed at `GET /api/election/total-proof`.

B. Improvement II: Threshold Master Secret Key

Limitation in the paper. The entire coercion resistance of zkVoting depends on the secrecy of msk. A single authority holding msk can: (a) identify real vs. fake keys by nullifying commitments; (b) unilaterally break coercion resistance if compromised. The paper acknowledges a "trusted tallier" assumption but provides no mechanism to distribute this trust.

Our implementation. We implement (t, n)-Shamir Secret Sharing [2] over $\mathbb{Z}_q$:

Split. Choose random polynomial over $\mathbb{Z}_q$:

$$f(x) = msk + a_1 x + \cdots + a_{t-1} x^{t-1}$$

Share $i = (i,\, f(i))$ for $i = 1, \ldots, n$.

Reconstruct. Lagrange interpolation over $\mathbb{Z}_q$ from any t shares recovers $f(0) = msk$.

Feldman VSS. Public commitments $\{C_j = a_j \cdot \mathsf{Base8}\}_{j=0}^{t-1}$ allow each authority to verify its own share without trusting the dealer:

$$s_i \cdot \mathsf{Base8} = \textstyle\sum_{j=0}^{t-1} C_j \cdot i^j$$

Threshold tally. Authority i computes partial decryption $P_i = s_i \cdot C_1$ for each ballot. t partial decryptions combine via Lagrange interpolation to recover $msk \cdot C_1$ without any party ever holding msk alone.

Exposed at `POST /api/election/init?t=2&n=3` and `POST /api/election/threshold-tally`.

C. Improvement III: Cross-Election Unlinkability

Limitation in the paper. The paper's serial number formula $sn = H(e, ck, sk_{\text{id}})$ uses the raw casting key ck. If a voter reuses the same real key in a second election (a natural scenario, e.g. if re-registration is not required), an observer can correlate serial numbers across elections and determine that the same voter participated in both. This breaks voter anonymity across elections.

Our implementation. We introduce election-scoped key derivation:

$$ckHash_e = H(ckHash,\, e)$$

The serial number becomes $sn = H(e, ckHash_e, sk_{\text{id}})$. Even if ck is identical across elections, $ckHash_e$ is election-specific, and the resulting serial numbers are uncorrelated. The Merkle leaf also uses $ckHash_e$, so eligibility proofs are election-scoped. This is a lightweight change: one additional Poseidon hash call per registration, zero circuit constraint overhead (the hash is computed off-circuit and the hash value is an existing circuit input).

D. Improvement IV: Forward Secrecy via Ephemeral Epoch Secrets

Limitation in the paper. The voter's identity secret sk_{id} is a permanent credential. If it is ever compromised—through device theft, malware, or legal compulsion—the attacker can compute the serial number $sn = H(e, ck, sk_{\text{id}})$ for *any* past or future election. This enables retroactive deanonymisation: the

attacker links all of the voter's historical ballots to their identity. The paper provides no mechanism for forward secrecy.

Our implementation. We add a per-election *ephemeral secret* σ_e to the serial number derivation:

$$sn = H(e,\ ckHash_e,\ sk_{\text{id}},\ \sigma_e)$$

The voter generates σ_e fresh for each ballot and *deletes it immediately after voting*. The circuit change is minimal: the serial number hasher moves from $\text{Poseidon}(3)$ to $\text{Poseidon}(4)$, with σ_e as an additional private input.

Security argument.

- If sk_{id} leaks *after* σ_e is deleted, the attacker cannot recompute past serial numbers (Poseidon preimage resistance).
- The public key $pkid = H(sk_{\text{id}})$ is unchanged, so voter identity binding and Merkle eligibility are unaffected.
- The cost is one additional Poseidon hash input per proof — negligible compared to the Merkle path verification.

Exposed in the circuit via `signal input epochSecret` and computed in the backend at vote-casting time.

E. Improvement V: Registration Censorship Resistance

Limitation in the paper. The authority controls which voters enter the Merkle tree rt. A malicious authority can silently exclude a voter's $(ckHash, pkid)$ pair. The voter's ZK proof will fail (Merkle membership check), and there is no mechanism to prove the exclusion was censorship rather than a legitimate rejection.

Our implementation. We introduce a *commit-reveal registration* protocol:

1) **Commit.** Before registration, the voter publishes a commitment:

$$regCommit = H(ckHash,\ pkid,\ nonce)$$

to a public append-only log.

2) **Reveal.** The voter reveals $(ckHash, pkid, nonce)$ during normal registration. The authority verifies the hash matches $regCommit$ and inserts into the Merkle tree.
3) **Audit.** Any observer can compare the commit log against the Merkle tree entries. A valid commit with no corresponding Merkle entry constitutes *cryptographic evidence of censorship*.

This makes registration censorship *publicly detectable* with zero false positives. The commit log is append-only (JSON file in prototype; a blockchain transaction in production).

Exposed at `POST /api/register/commit` and `GET /api/register/audit`.

F. Improvement VI: Re-Vote Grace Period

Limitation in the paper. Coercion resistance requires the voter to re-vote with their real key after being coerced. A sophisticated coercer can time the attack to $T - 1\,\text{min}$ before the public voting deadline, leaving the voter no window to re-vote. The paper assumes synchronous phases but does not address timing-based coercion attacks near phase boundaries.

Our implementation. We introduce a *hidden grace period* after the public voting deadline:

- The election has two deadlines: T_{public} (announced) and T_{grace} (secret, known only to the system).
- $T_{\text{grace}} > T_{\text{public}}$ (default: 60 minutes).
- During the grace window $[T_{\text{public}}, T_{\text{grace}})$, re-votes with the *same serial number* overwrite the previous ballot (last-write-wins). New first-time votes are rejected.
- The coercer believes the election ended at T_{public}; the voter can secretly re-vote during the grace window.
- T_{grace} is never exposed via the public API.

This closes the timing attack vector at the cost of a brief (configurable) delay before tally publication.

Exposed at `POST /api/election/init` with the `graceMinutes` parameter.

VI. Systematic Assumption Analysis

We conducted a systematic audit of the security assumptions made by the zkVoting paper. We identified 13 assumptions, categorised by severity and real-world impact. Table I summarises all 13; those addressed by our improvements are marked.

Of the 13 assumptions, our six improvements directly address five (A3, A4, A6, A8, A11). Section X discusses proposed solutions and rationale for the remaining eight unaddressed assumptions.

VII. Comparative Analysis

A. Security Properties

Table II compares the security properties of the paper's original protocol, the basic prototype, and our enhanced system.

B. Protocol Components Implemented

Table III shows which protocol components are present.

C. Circuit Constraint Comparison

Table IV compares circuit constraint counts. The paper uses MiMC7 hashing and includes a full hybrid-encryption proof in R_{vote}; we use Poseidon (more ZK-efficient) and handle encryption off-circuit (symmetric AES-GCM), so our circuit is substantially smaller.

The dramatic reduction in R_{vote} constraints (from 15,829 to ≈1,450) is because: (a) Poseidon has ≈65 constraints per 2-input call vs. MiMC7's ≈220+; (b) we do not prove the hybrid-encryption computation in-circuit (a pragmatic simplification).

D. Tally Complexity

The threshold tally introduces a factor of t (the number of authorities required) but t is a small constant independent of n_b.

TABLE I
SYSTEMATIC ASSUMPTION ANALYSIS OF THE ZKVOTING PROTOCOL

ID	Assumption	Severity	If Violated	Addressed
A1	No on-chain Sybil prevention	Critical	One person, many votes	—
A2	Private untappable channel	Critical	Coercion resistance fails	—
A3	Re-vote timing near deadline	High	Timing-based coercion wins	Impr. VI
A4	Authority registration fairness	Critical	Ballot stuffing / censorship	Impr. I+V
A5	Groth16 trusted setup	High	Fake proofs possible	—
A6	Threshold collusion ($< t$ honest)	Critical	Full msk reconstructed	Impr. II
A7	DDH/DLP quantum hardness	Medium	All EC crypto broken	—
A8	No forward secrecy on sk	High	Retroactive deanonymisation	Impr. IV
A9	Poseidon random oracle model	Medium	Commitment binding may fail	—
A10	Blockchain as bulletin board	High	Tampering with tallied votes	—
A11	Cross-election key linkability	Medium	Voter tracking across elections	Impr. III
A12	Synchronous phase boundaries	Low	Late votes rejected	—
A13	DLP-based tally $O(n)$ scaling	Low	Large-scale elections impractical	—

TABLE II
SECURITY PROPERTY COMPARISON

Property	Paper	Basic proto.	Ours
Ballot privacy (BP)	✓	✓	✓
Voter anonymity (VA)	✓	✓	✓
Receipt freeness (RF)	✓	×	✓
Coercion resistance (CR)	✓	×	✓
Individual verif. (IV)	✓	✓	✓
Universal verif. (UV)	✓	partial	✓
E2E verifiability (E2EV)	✓	×	✓
Eligibility verif. (EV)	✓	×	✓
π_{total} proof	defined only	×	✓
Threshold msk	×	×	✓
Cross-election unlinkability	×	×	✓
Forward secrecy (Impr. IV)	×	×	✓
Reg. censorship detect. (V)	×	×	✓
Re-vote grace period (VI)	×	×	✓

TABLE III
PROTOCOL COMPONENT COVERAGE

Component	Paper	Basic proto.	Ours
NC scheme (Constr. 1)	defined	×	✓
Real/fake key issuance	defined	×	✓
KeyProve	defined	×	✓
SimKeyProve (deniability)	defined	×	✓
Merkle key set (rt)	defined	×	✓
R_{id}: $pkid = H(sk)$	in circuit	×	✓
Merkle proof in circuit	in circuit	×	✓
Nullification tally O(n)	defined	×	✓
π_{total} proof	defined	×	✓
Threshold msk (SSS)	×	×	✓
Cross-election unlinkability	×	×	✓
Forward secrecy (σ_e)	×	×	✓
Commit-reveal registration	×	×	✓
Hidden grace period	×	×	✓

TABLE IV
CIRCUIT CONSTRAINT COMPARISON (APPROXIMATE)

Relation	Paper (MiMC7)	Ours (Poseidon)
R_{id}	365	≈ 58
R_{total}	2,243	DLEQ Schnorr (no circuit)
R_{vote}	15,829	$\approx 1{,}450$[a]
R_{tally}	6,111	off-circuit EC ops

[a]Excludes encryption proof; includes Merkle depth-16.

E. Trust Assumptions

VIII. IMPLEMENTATION ARCHITECTURE

A. Technology Stack

- **ZK proving**: Circom 2.1.6, snarkjs 0.7.6 (Groth16)
- **EC arithmetic**: circomlibjs 0.1.7 (Baby Jubjub, Poseidon)
- **Backend**: Node.js 20, Express 5
- **Persistence**: JSON files (prototype; production would use a tamper-evident ledger)

B. Key New Files

- `nullifiableCommitment.js`: Construction 1 (setup, keyGen, commit, nullify, open, openNull, keyProve, simKeyProve)
- `merkleTree.js`: depth-16 Poseidon Merkle tree with proof generation and bulletin-board persistence
- `totalProof.js`: π_{total} accumulation and DLEQ Schnorr proof
- `thresholdMsk.js`: Shamir SSS, Feldman VSS, partial nullification
- `registrationAudit.js`: commit-reveal registration with censorship audit (Improvement V)
- `enhanced_vote.circom`: full R_{vote} circuit with pkid proof, Merkle membership, and epochSecret (Improvement IV)

C. New API Endpoints

- `POST /api/admin/reset` — wipe data for fresh experiment (Dashboard)

TABLE V
TALLY TIME COMPLEXITY

System	Complexity	Notes
Civitas/JCJ	$O(n_b^2)$	PET test for each pair
VoteAgain	$O(n_b \log n_b)$	sort-based
Loki	$O(n_v)$	per-voter re-randomization
zkVoting (paper)	$O(n_b)$	nullification
Ours (basic)	$O(n_b)$	nullification
Ours (threshold)	$O(t \cdot n_b)$	t partial nullifications

TABLE VI
TRUST ASSUMPTION COMPARISON

System	Trust required for CR
JCJ/Civitas	registrar + tallier
VoteAgain	registrar + voting server + tallier
zkVoting (paper)	registrar + tallier (single msk holder)
Ours (Improvement II)	any t of n talliers (threshold)

- `GET /api/admin/voters` — full voter registry (Dashboard)
- `POST /api/register/fake-key` — issue fake casting key
- `POST /api/register/sim-key-prove` — SimKeyProve for deniability
- `POST /api/election/init` — NC setup + threshold msk split
- `GET /api/election/total-proof` — π_{total} proof
- `POST /api/election/threshold-tally` — threshold reconstruction demo
- `GET /api/bulletin-board` — full casting-key set + Merkle root
- `GET /api/results/nullify/:ballotId` — per-ballot nullification audit
- `GET /api/results/legacy-tally` — original tally for comparison
- `POST /api/register/commit` — registration commitment (Improvement V)
- `GET /api/register/audit` — censorship detection audit (Improvement V)

D. Research Simulation Dashboard

To facilitate protocol evaluation and simulate diverse election scenarios, we developed a comprehensive web-based dashboard using vanilla web technologies. Designed for researchers, the interface abstracts away the underlying API complexity, allowing users to:

- **Manage Elections:** Initialize elections with tunable parameters (threshold bounds, authority count, grace period).
- **Simulate Cohorts:** Perform batch voter registration and automated ballot casting to quickly populate the system state.
- **Inspect Data:** Browse the public bulletin board (Poseidon Merkle tree), encrypted ballot log, and raw backend state files in real time.
- **Test Coercion Resistance:** Issue fake casting keys and generate simulated deniability proofs (SimKeyProve) interactively.
- **Execute Audits:** Trigger complex cryptographic events such as π_{total} verification, multi-authority threshold tallies, and nullification-based censorship audits with a single click.

The dashboard maintains a real-time protocol log, capturing every cryptographic operation, API request, and state transition, thereby significantly simplifying the evaluation of the protocol's E2E verifiability and coercion-resistance guarantees.

IX. SECURITY ANALYSIS OF IMPROVEMENTS

A. π_{total} Soundness

The DLEQ Schnorr proof is sound under the Discrete Log assumption: a computationally bounded prover cannot produce a valid (A_1, A_2, k, c) satisfying both verification equations without knowing msk such that $msk \cdot g_1 = g_4$. Together with the nullifiability property of Π_{NC}, this proves $n_v^{\text{real}} = n_v$ with negligible soundness error.

B. Threshold Security

By the information-theoretic security of Shamir SSS over $\mathbb{Z}_q$, any $t-1$ shares reveal zero information about msk. Feldman VSS adds verifiability: a dishonest dealer distributing inconsistent shares is detected with probability 1 (unconditional, not just computational). The threshold partial-nullification protocol is secure as long as at least t of the n authorities are honest and do not collude.

C. Cross-Election Unlinkability

The derivation $ckHash_e = H(ckHash, e)$ is election-specific. Under the collision resistance of Poseidon (modelled as a random oracle in the BN128 field), $ckHash_{e_1}$ and $ckHash_{e_2}$ for $e_1 \neq e_2$ are computationally independent, so no PPT adversary can link the same voter's serial numbers across elections with more than negligible advantage.

X. LIMITATIONS AND FUTURE WORK

1) **Trusted setup (A5)**: Groth16 requires a per-circuit trusted setup. Migrating to Plonk or Halo2 would eliminate this assumption. *Not implemented*: requires a fundamentally different ZK backend; circuit conversion is thesis-level effort.
2) **Blockchain bulletin board (A10)**: the prototype uses JSON files. A tamper-evident ledger (e.g. an Ethereum smart contract) is required for production deployment and full E2E verifiability. *Not implemented*: engineering effort orthogonal to cryptographic contributions; well-understood integration path.
3) **Sybil prevention (A1)**: the protocol lacks on-chain identity verification. *Proposed solution*: bind

each Merkle leaf to a unique credential hash $credHash = H(national_id, e)$, rejecting duplicate credential hashes. Privacy is preserved via the hash. *Not implemented*: requires integration with an external identity provider and raises policy questions beyond the cryptographic scope of this work.

4) **Private channel assumption (A2)**: coercion resistance assumes the voter has an "untappable" private channel for re-voting. This is a physical-world constraint that no cryptographic protocol can fully solve. Practical mitigations include anonymous communication networks (Tor), in-person re-vote kiosks, or time-delayed re-vote submission. *Not implemented*: inherently non-cryptographic.
5) **Quantum resistance (A7)**: all security reductions depend on DDH/DLP hardness over Baby Jubjub. A sufficiently powerful quantum computer breaks these assumptions. *Proposed solution*: migrate to lattice-based commitments and hash-based ZK (e.g. STARKs). *Not implemented*: post-quantum ZK is an active research frontier with no production-ready toolchain compatible with Circom.
6) **Poseidon random oracle model (A9)**: the security proofs model Poseidon as a random oracle. If algebraic attacks exploit Poseidon's structure, commitment binding and collision resistance may degrade. *Not implemented*: would require either formal cryptanalysis or a hash-function swap (e.g. to Rescue), both outside the scope of a systems-level contribution.
7) **Synchronous phases (A12)**: the paper assumes strict phase boundaries. Late arrivals to the registration or voting phase are excluded. Improvement VI partially addresses this for the voting phase; registration-phase flexibility remains future work.
8) **Tally scaling (A13)**: the $O(n)$ nullification tally uses Baby Jubjub EC scalar multiplication per ballot. For national-scale elections ($> 10^7$ voters), this may require parallelisation or batch verification optimisations.
9) **Encryption proof in circuit**: we prove only the Poseidon commitment in-circuit; the hybrid encryption $CT = \Pi_{\mathrm{HBE}}.\mathsf{Enc}(pk, m; k)$ is not circuit-verified. Adding this would restore the paper's full R_{vote} constraint set.
10) **Client-side proof generation**: proof generation should run on the voter's device (browser/mobile) to preserve the ZK property; our prototype performs it server-side for simplicity.

XI. Conclusion

We presented a complete implementation of the zkVoting protocol's missing cryptographic components and identified six novel improvements. The nullifiable commitment scheme (Construction 1), implemented over Baby Jubjub using circomlibjs, correctly satisfies hiding, binding, nullifiability, key indistinguishability, and key deniability. The enhanced Circom circuit adds voter public-key proofs and Merkle membership checks. The O(n) nullification tally correctly discards fake-key ballots.

Beyond completing the paper, we showed that the π_{total} eligibility proof can be instantiated concretely using a DLEQ Schnorr argument over the aggregate homomorphic commitment—preventing ballot stuffing without a full Groth16 circuit. Threshold Shamir SSS eliminates the single-authority trust bottleneck for the master secret key. Election-scoped key derivation closes the cross-election linkability gap at negligible cost. Ephemeral epoch secrets provide forward secrecy against retroactive deanonymisation. Commit-reveal registration makes authority censorship publicly detectable. A hidden grace period closes timing-based coercion attack vectors.

Our systematic assumption analysis identified 13 security assumptions in the original protocol, of which our six improvements address five. For the remaining eight, we provide proposed solutions and justify their exclusion from the current prototype.

These contributions together produce a prototype that satisfies all eight security properties in Table II and surpasses the original zkVoting paper on six additional dimensions.

Responsible Disclosure of AI Assistance

In the interest of academic transparency, the authors disclose the use of Large Language Models (LLMs) and agentic AI tools during the development of this project. AI assistance was utilized primarily to accelerate the rapid prototyping of the web dashboard (HTML/CSS layout generation and DOM scripting), scaffolding the Express.js backend boilerplate, and to keep documentations.

All core cryptographic logic, including the circomlibjs nullifiable commitment implementations, the zero-knowledge circuit modifications in `enhanced_vote.circom`, and the theoretical assumption analysis were meticulously designed, directed, and mathematically checked by the primary authors. The inclusion of AI-assisted scripting strictly served to expedite empirical validation and UI construction without compromising the academic integrity or theoretical rigor of the core research.

References

[1] S. Park, J. Choi, J. Kim, and H. Oh, "zkVoting: Zero-knowledge proof based coercion-resistant and E2E verifiable e-voting system," *Cryptology ePrint Archive*, Paper 2024/1003, 2024.

[2] A. Shamir, "How to share a secret," *Commun. ACM*, vol. 22, no. 11, pp. 612–613, Nov. 1979.

[3] A. Juels, D. Catalano, and M. Jakobsson, "Coercion-resistant electronic elections," in *Proc. ACM Workshop Privacy Electron. Soc.*, 2005, pp. 61–70.

[4] J. Groth, "On the size of pairing-based non-interactive arguments," in *Proc. EUROCRYPT*, 2016, pp. 305–326.

[5] M. Bellés-Muñoz, B. Whitehat, and J. Baylina, "Twisted Edwards elliptic curves for zero-knowledge circuits," *Mathematics*, vol. 9, no. 23, 2021.

[6] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, and M. Schofnegger, "Poseidon: A new hash function for zero-knowledge proof systems," in *Proc. USENIX Security*, 2021.

[7] W. Lueks, I. Querejeta-Azurmendi, and C. Troncoso, "VoteAgain: A scalable coercion-resistant voting system," in *Proc. USENIX Security*, 2020, pp. 1553–1570.

[8] P. Feldman, “A practical scheme for non-interactive verifiable secret sharing,” in *Proc. FOCS*, 1987, pp. 427–438.